PROJECT REPORT

WWLLM Lecture Studio

Turning Indian-language lectures into transcripts, notes, quizzes and translations



Tamish Sridatta Billa

USN 1RVU23CSE498

October 2026

1. Summary

WWLLM Lecture Studio takes a recorded lecture and produces a timed transcript, structured notes, a practice quiz and translations into ten Indian languages. Every output links back to the second of audio it came from, so a student can click a note or a quiz answer and hear the lecturer say it.

It runs open models on an ordinary laptop with no GPU. The 98-second sample lecture takes about 31 minutes from audio to finished translations. Results are stored by content, so a lecture only has to be processed once, and the stored sample works on a machine with no models installed.

0.258	4	10	122
average word error rate across 40 lectures	spoken languages tested	translation languages	automated tests passing

2. How it works

Audio or video Upload, or the sample › **Silero-VAD** Finds each stretch of speech › **Whisper** Speech to text › **Param-1** Notes and quiz › **NLLB-200** Ten Indian languages

Each stage tries three things in order: a stored result for this exact input, the real model, then an offline method. A badge on every result says which one produced it, so offline output is never mistaken for model output.

Stage	Model	Job	Time on the sample
Speech detection	Silero-VAD v5	Cuts the audio into 26 stretches of speech with timestamps	7 s
Transcription	whisper-hindi-small	Hindi speech to text, one stretch at a time	2.0 min
Notes	BharatGen Param-1 2.9B	Structured English notes, streamed as they are written	9.6 min
Quiz	BharatGen Param-1 2.9B	Five multiple-choice questions with explanations	8.2 min
Translation	NLLB-200	Notes and quiz into one language, keeping the answer key	64 s

Measured on an Intel i5-1235U with 16 GB of RAM and no GPU.

IndicConformer and IndicTrans2 from AI4Bharat were the original choices for speech and translation. Both are gated on Hugging Face and need a personal login. The code tries them first when their toolkits are installed, and otherwise uses the open models above, which produced every result in this report.

Search across scripts

Indian lectures mix languages, so a student may search for a word in a different script from the one the lecturer's transcript uses. Search needs no model. Hindi, Tamil and the other major Indic scripts share one Unicode layout, so a single table reduces them, and Latin spelling, to the same consonant skeleton:

```
gradient  →  K R T N T      ग्रेडिएंट  →  K R T N T      கிரேடியன்ட்  →  K R T N T
```

3. The application in use

Every screenshot comes from the running application and shows real model output.

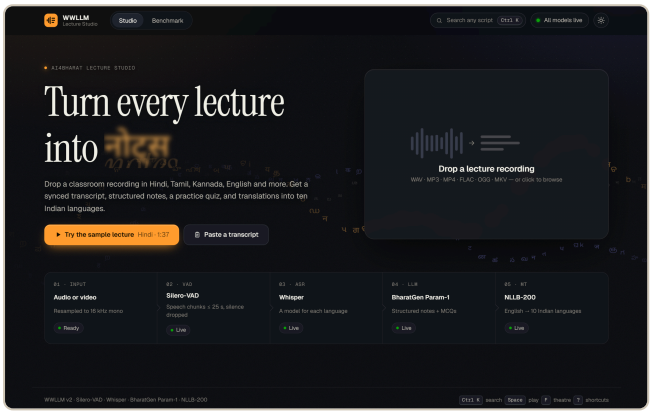


Figure 1. The home page. Upload a recording, paste a transcript, or open the sample lecture.

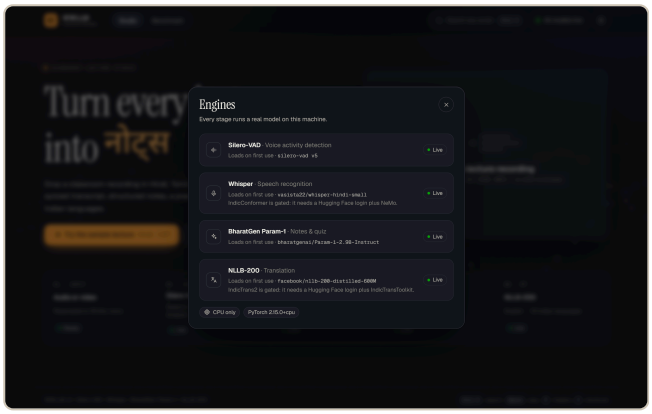


Figure 2. The engines panel shows the model behind each stage and whether it can run on this machine.

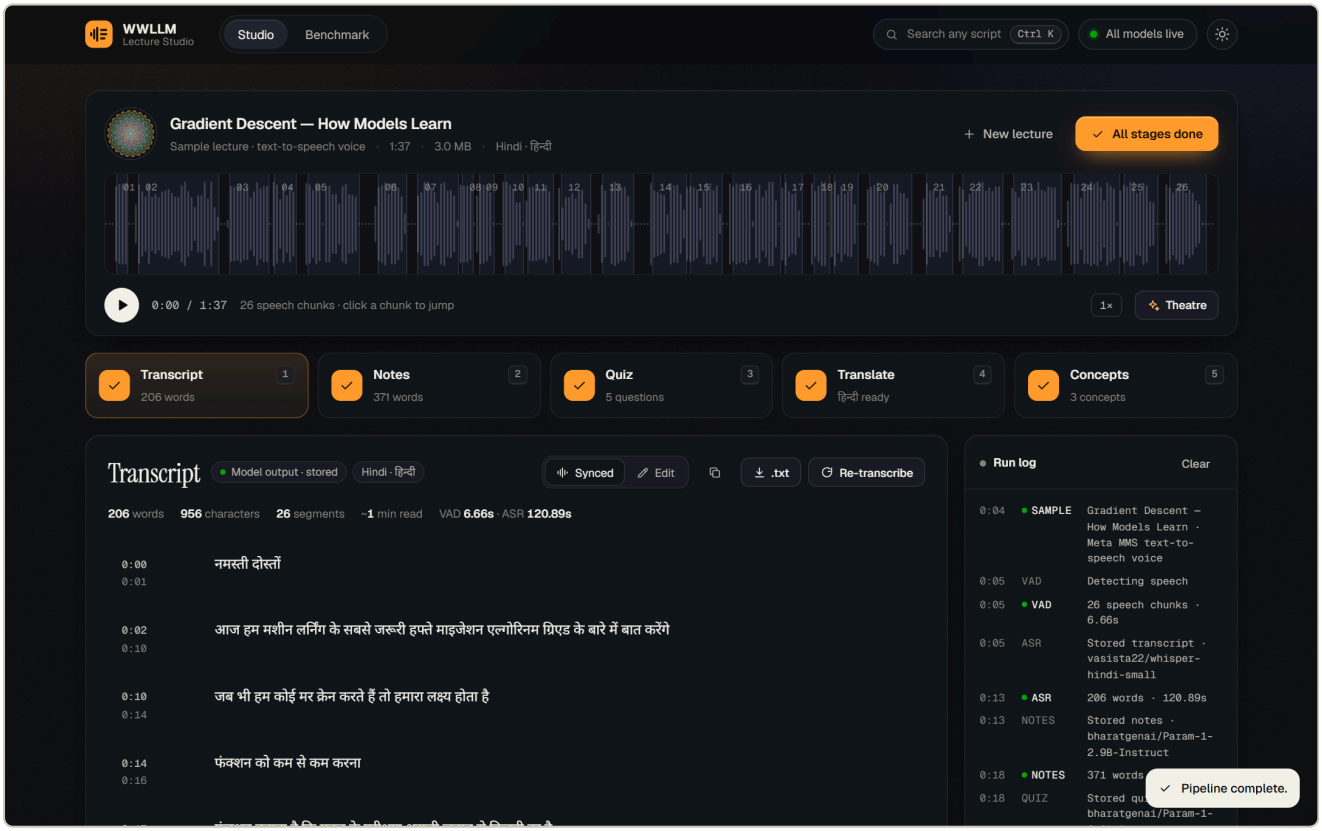


Figure 3. The transcript from Whisper, 206 words of Hindi across 26 stretches of speech. Words light up during playback, and clicking one jumps to that moment.



Figure 4. Notes from Param-1, which read the Hindi transcript and wrote 371 words of English. Each section has a chip that plays the matching part of the lecture.

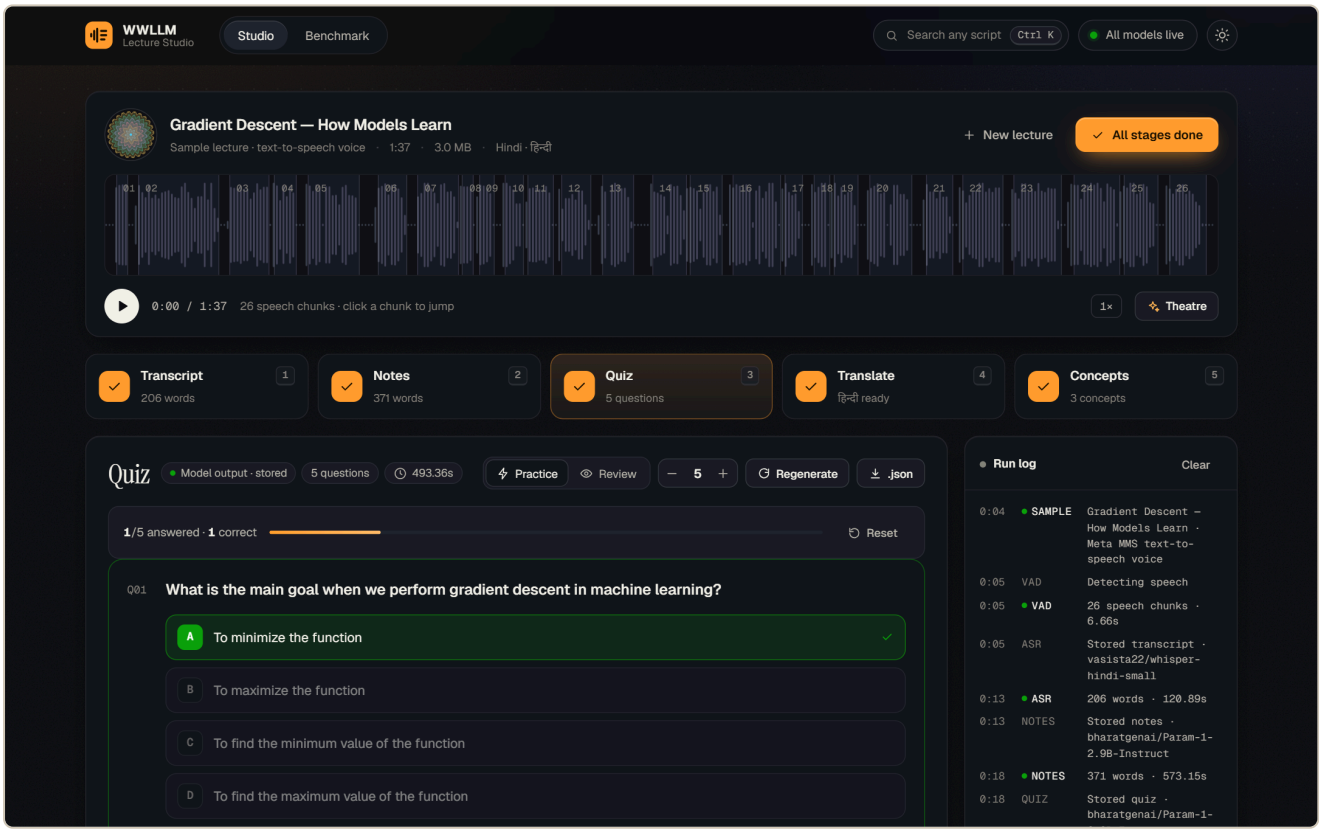


Figure 5. The quiz, written by Param-1. After an answer, the student can replay the moment in the lecture that holds it.

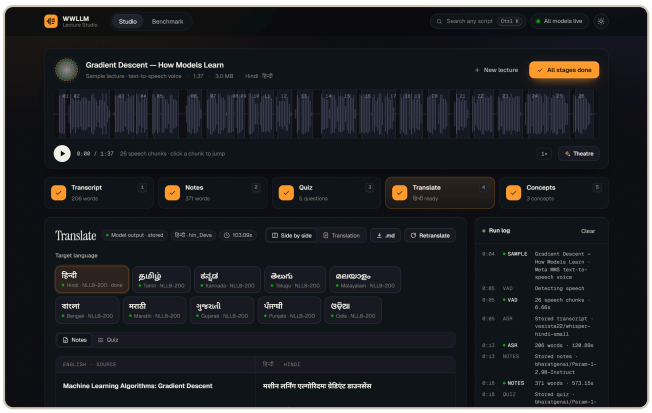


Figure 6. Notes and quiz translated into Hindi by NLLB-200. Headings, lists and the answer key are kept.

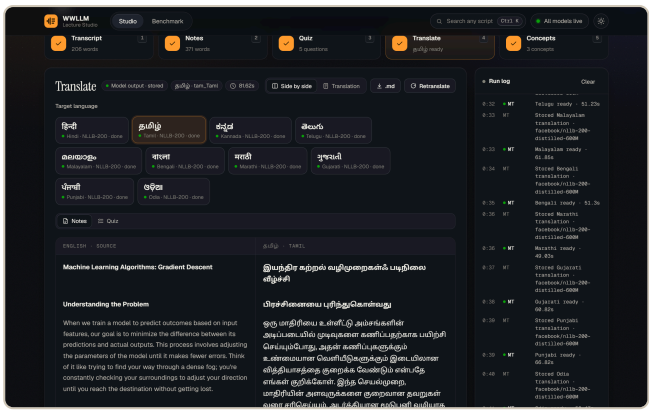


Figure 7. The same notes in Tamil, side by side with the English.

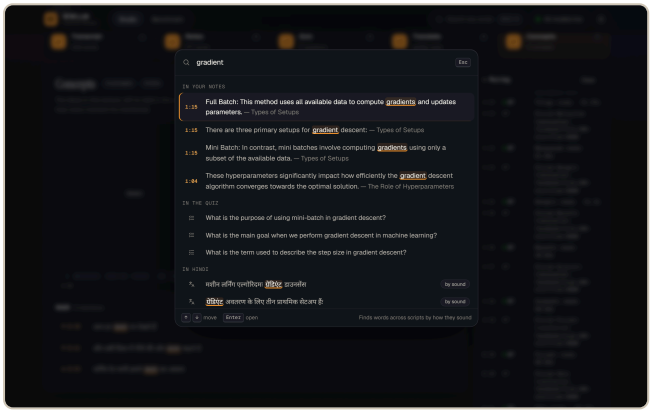


Figure 8. Ctrl K search. The English word "gradient" finds matches in the notes, the quiz and the Hindi translation.

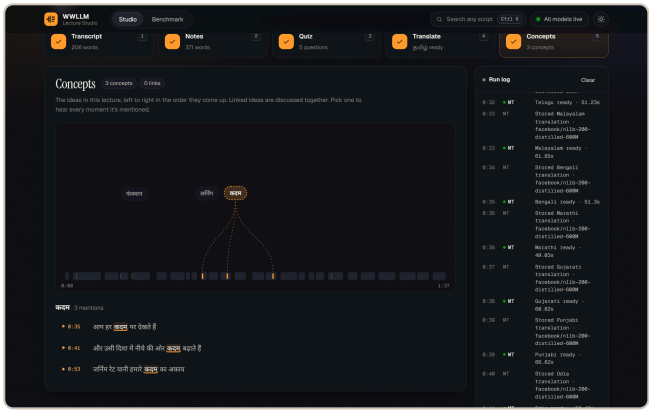


Figure 9. The concept map places each idea on the timeline and lists every moment it is mentioned.



Figure 10. Theatre mode. A rangoli shaped by the loudness of the speech draws itself during playback, with word-by-word captions.

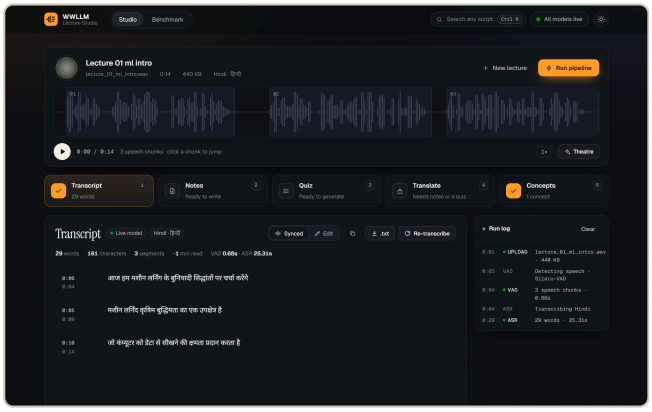


Figure 11. A new recording uploaded and transcribed live by Silero-VAD and Whisper.

4. Results

The benchmark has 40 short technical lectures. Meta's MMS text-to-speech voices read each reference text aloud, then the real pipeline transcribed the audio, and the output was scored against the text. Word error rate (WER) counts the words changed, dropped or added, as a share of the reference. Punctuation and case are ignored.

Language	Model	Lectures	Average WER
English	openai/whisper-small	22	0.257
Hindi	vasista22/whisper-hindi-small	14	0.277
Tamil	vasista22/whisper-tamil-small	2	0.225
Kannada	vasista22/whisper-kannada-small	2	0.167
All		**40**	**0.258**

28 lectures pass the 0.30 threshold and 12 fail. This is clean, machine-read speech, so a real classroom with echo and noise is likely to score worse. The Tamil and Kannada rows rest on two clips each, which is too few to compare languages.



Figure 12. The benchmark page, built from this measurement.

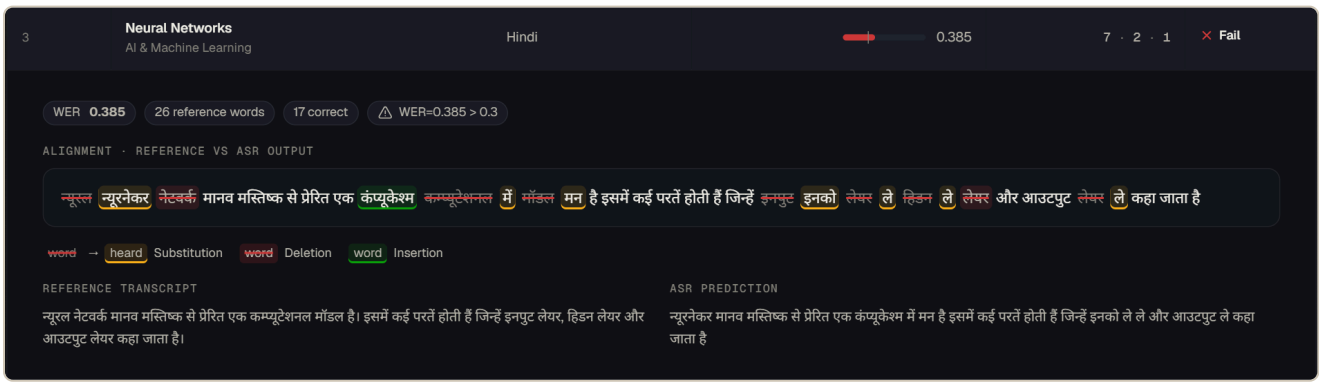


Figure 13. Any lecture opens into a word-by-word comparison of the reference and what the model heard.

Quality of the output

On the sample lecture, Whisper scored a WER of 0.27. Its typical mistake is a technical loanword: "learning rate" spoken in Hindi came out as "जर्निंग रेट".

Param-1 recovered the structure of the lecture from that imperfect transcript and named the topic correctly. It also adds general knowledge the lecturer never said, so its notes need a human read before they are trusted.

The quiz is on topic but uneven. Of the five questions, the first has two options that both mean the right answer, the third has a poor answer key, and three have no explanation, so a teacher should review it before using it.

NLLB-200 gives readable translations but garbles technical terms. "Gradient Descent" became ग्रेडिएंट डाउनसेंस in Hindi, close in sound but not the real term.

5. Problems found and fixed

Running real models on real speech exposed these faults.

Symptom	Cause	Fix
Models used one CPU core	Importing silero_vad pins the whole process to one thread	Restore the thread count after the import
Param-1 crashed on load, then wrote gibberish	Its custom code predates transformers 5, which left the rotary position tables uninitialised	Reset the config and rebuild the tables after loading
Each new word re-read the whole prompt	generate() can't use the model's cache format	A small sampling loop that carries the cache itself
Whisper kept 80 words of a 206-word lecture	The fine-tuned model stops at the first pause, and chunks held several sentences	One chunk per stretch of speech, which took the sample's WER from 0.77 to 0.27
Notes had no contents list or section chips	Param-1 wrote #### headings, and the interface looked for ##	Level the heading depth on display
The quiz failed three ways	Param-1 wrote broken JSON, copied the example, then stopped after one question	A plain text format, with each question started for the model
The benchmark was simulated	Hand-written predictions scored against sine-wave audio	Real speech for all 40 clips and a real measurement

6. Testing

122 automated tests pass: 87 in Python, covering the pipelines, service layer, HTTP API and Streamlit app, and 35 in JavaScript, covering search, anchors and concepts. Without the ML stack installed, the Python suite still passes, with 16 model tests skipped. A scripted run in Chrome then went through the sample lecture, all ten translations, search, Theatre, the benchmark page and a live upload. All 21 checks passed, with no console or network errors.

7. Limits

- Param-1 is slow without a GPU, at about 9.6 min for one set of notes.
- IndicConformer and IndicTrans2 have not been run or measured, because they need a Hugging Face login.
- The user picks the spoken language. There is no automatic detection, and the wrong choice produces nonsense.
- The accuracy figures come from synthetic speech, so they show the pipeline working and don't predict classroom accuracy.
- Word timing is estimated from chunk timing, so karaoke highlighting is close but not exact.
- MMS voices and NLLB-200 are CC-BY-NC, which rules out commercial use.

8. Running it

```
pip install -r requirements.txt
python server.py --open
```

Open `http://127.0.0.1:8000` and press Try the sample lecture. Models download on first use, about 8 GB in all. With `requirements-web.txt` alone, the studio still runs the sample from stored results.